



Datenbanken (DAB4)

Elasticsearch

24.05.26

Fabian Kopf (2410824006)

SS 26

1. Contents

2.	Introduction	3
2.1.	What is Elasticsearch.....	3
3.	Fundamentals of Elasticsearch	4
3.1.	NoSQL Document Store and search engine	4
3.2.	When to use Elasticsearch	5
4.	Architecture	6
4.1.	Core Concepts	6
4.2.	Inverted Index.....	7
4.3.	Sharding, Replication and Distributed Search.....	9
5.	Key features.....	10
5.1.	Full-Text Search and Relevance Scoring	10
5.2.	Distributed and Scalable.....	10
5.3.	Near Real-Time Data Analysis.....	10
5.4.	RESTful API	11
6.	Query Language	12
6.1.	Query DSL.....	12
6.2.	Aggregations.....	13
7.	Project.....	14
7.1.	Project Description	14
7.2.	The Indexing script: fl_dataset_inserter_py	14
7.3.	The Query Script: fl_queries.py	15
7.4.	Setup and Configuration.....	15
7.5.	Kibana Dashboard	17
8.	Conclusion.....	20
9.	References	21

2. Introduction

The amount of data that modern application generate has grown to a scale where traditional relational databases often struggle to keep a decent performance. Searching through these large datasets (which are often unstructured text), analysing log streams in real time etc. is inefficient in such systems. Analysing and querying high volume datasets should normally be done in a few milliseconds. A dedicated search and analytics engine is needed for these scenarios. Elasticsearch is such an engine and has become one of the most used solutions for this purpose.

2.1. What is Elasticsearch

Elasticsearch is an open-source, distributed search and analytics engine that was first released in 2010. It is developed and maintained by Elastic NV and is built on top of Apache Lucene. Apache Lucene is a Java-based library that provides the core indexing and full-text search capabilities. While Lucene handles the low-level mechanics of searching, Elasticsearch adds a distributed architecture, a REST API, cluster management and a query language on top of it.

Elasticsearch stores data as JSON documents inside structures called indices. Unlike relational databases, it does not require a rigid schema to be defined upfront. Because of that fact it is obvious that Elasticsearch is a NoSQL Database. Due to its architecture and schemaless nature, documents with varying structures can be indexed seamlessly, while Elasticsearch automatically infers the data types when no explicit mapping is defined. This flexibility makes it well suited for rapidly changing data structures and unstructured content such as log entries or free-form text.

Elasticsearch is part of the Elastic Stack which consists of four components: Elasticsearch, Kibana, Logstash and Beats. Elasticsearch itself is the storage and search product of this stack. Kibana provides a web-based interface for visualizing data stored in Elasticsearch. With Kibana it is easy to create charts, maps and dashboards. Logstash is a data processing pipeline that can ingest data from different sources, transform it and forward it to Elasticsearch. Beats are lightweight agents that run on servers and ship data directly to Elasticsearch or Logstash. For this paper the focus is on the functionality of Elasticsearch and Kibana.

3. Fundamentals of Elasticsearch

3.1. NoSQL Document Store and search engine

As said before, Elasticsearch belongs to the category of NoSQL databases and more specifically to the subcategory of search engines. Unlike relational databases, which organize data into table with fixed columns and rows, Elasticsearch stores data as JSON documents. Each document is a self-contained object that can have its own structure, and no two documents are required to have identical fields.

Elasticsearch has no concept of joins. Instead, related data must be combined into a single document before indexing. This process is called denormalization. This means that data is intentionally duplicated across documents in exchange for faster query performance.

Despite being a NoSQL document store, Elasticsearch is also a search engine which means it is specifically optimized for search and analytics workloads. The key difference to a general-purpose document database such as MongoDB is the underlying data structure. While MongoDB stores documents in a format optimized for retrieval by ID or field value, Elasticsearch builds an [inverted index](#) over the content of every document at indexing time. This inverted index maps every individual word to the documents that contains it. This allows Elasticsearch to answer full-text search queries in milliseconds regardless of the total number of documents. This makes Elasticsearch orders of magnitude faster for search-heavy workloads compared to a relational or document database that would need to scan through records sequentially to find matches.

Aspect	Elasticsearch	Relational Database
Data model	Document-oriented, JSON	Table-based, rows and columns
Schema	Flexible, optional	Rigid, predefined
Query language	Query DSL	SQL
Joins	Not supported	Supported
Scalability	Horizontal	Vertical
Consistency	Eventual	Strong (ACID)
Best suited for	Search and analytics	Transactions and integrity

3.2. When to use Elasticsearch

Elasticsearch is not a “general-purpose” database and is not designed to replace relational systems in all scenarios. It is best suited for specific use cases where search speed, analytical flexibility or the ability to handle large volumes of unstructured data are the primary requirements.

Full-text search is the most common use case. Applications like e-commerce platforms, news portals, or documentation system need to deliver fast and relevant search results across large collections of text.

Log and event monitoring is another use case. Organizations running server infrastructure typically generate enormous volumes of log data continuously. By streaming these logs into Elasticsearch, engineers can search for specific error messages, detect anomalies and monitor system health in near real time.

Business analytics is also an area to use Elasticsearch. Its aggregation capabilities allow analysts to group, count, sum and average large datasets in milliseconds, producing results that would require complex SQL queries and significant processing time in a relational system.

Geospatial search is supported by Elasticsearch. The `geo_point` type stores latitude and longitude coordinates. This enables proximity queries, radius-based filtering, and map visualizations, making Elasticsearch suitable for applications such as delivery tracking and travel booking platforms

4. Architecture

4.1. Core Concepts

Before understanding how Elasticsearch distributes data and processes queries, it is first necessary to understand the underlying system architecture and how the individual components work together.

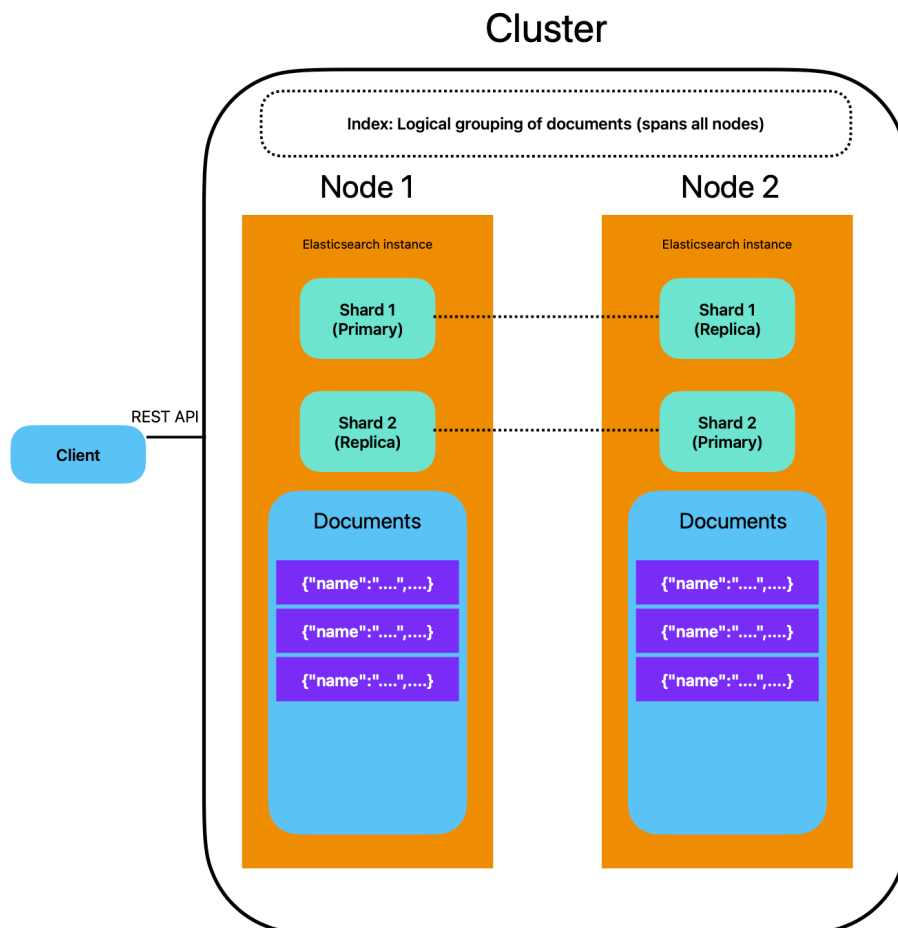


Figure 1: Based on description of [https://www.geeksforgeeks.org/elasticsearch-architecture/](https://www.geeksforgeeks.org/elasticsearch/elasticsearch-architecture/)

Cluster

A cluster consists of one or more nodes that work together and share the same data. Every cluster has a unique name that nodes use to identify to which cluster they belong to. In production environment a cluster typically spans multiple physical or virtual machines.

Node

A node is a single running instance of Elasticsearch. Each node stores a portion of the cluster's data and participates in indexing and search operations. Nodes can be assigned different roles. A master node manages cluster-level operations such as tracking of which nodes are part of. Data nodes are responsible for storing data and executing queries.

Index

An index is a logical container for a collection of documents that share a common purpose. It is divided into one or more shards, each of which is an independent storage unit that can be placed on any node in the cluster. The index is therefore a logical concept. In a single-node setup all shards are located on the same machine, while in a multi-node cluster the shards are distributed across different nodes.

Document

A document is the basic unit of data in Elasticsearch. It is represented as a JSON object and stored within an index. Every document has a unique identifier assigned. When a document is indexed, Elasticsearch analyses its fields and build the internal data structure needed for fast access.

Shard

A shard is a physical subdivision of an index. When an index is created, Elasticsearch distributes its documents across one or more shards. Each shard is itself a fully functional index and can be stored on any node in the cluster. Sharding serves two purposes. It allows data that exceeds the storage capacity of a single machine to be spread across multiple nodes. Furthermore, it enables search queries to be executed in parallel across all shards simultaneously.

Replica

A replica is a copy of a shard stored on a different node than the original, also called the primary shard. Replicas serve two purposes. They can act as a backup in case a node fails and they can serve read requests in parallel with the primary shard, increasing query throughput.

4.2. Inverted Index

When a document is indexed in Elasticsearch, its text fields are not stored as they are. The content is written to the inverted index. It passes through an analyser. The analyser is a processing pipeline that transforms raw text into a set of normalized tokens that can be efficiently looked up during search. The pipeline consists of three stages: Tokenization → lowercasing → word removal

Tokenization

The text is split into individual terms.

```
Doc 1: "Hamilton won the race"
Doc 2: "Verstappen won the championship"
Doc 3: "Hamilton lost the championship"
```

```
-> ["Hamilton", "won", "the", "race"]
-> ["Verstappen", "won", "the", "championship"]
-> ["Hamilton", "lost", "the", "championship"]
```

Lowercasing

Every token is converted to lowercase so that searches are case-insensitive by default.

```
-> ["Hamilton", "won", "the", "race"]
-> ["Verstappen", "won", "the", "championship"]
-> ["Hamilton", "lost", "the", "championship"]
```

Stop Word removal

Common words that appear in almost every document and carry no meaningful search value, such as "the", "a", or "and", are removed from the token list. These words are known as stop words.

```
["hamilton", "won", "race"]  
["verstappen", "won", "championship"]  
["hamitlon", "lost", "championship"]
```

Result

The same analyser pipeline is applied to the search query before it is looked up in the index. After the analyser pipeline, the tokens are stored like this:

```
"hamilton"      -> Doc 1, Doc 3  
"won"           -> Doc 1, Doc 2  
"race"          -> Doc 1, Doc 3  
"verstappen"    -> Doc 2  
"championship" -> Doc 2  
"lost"          -> Doc 3
```

For each token Elasticsearch stores in which document this token appears.

Search

Elasticsearch does not look for an exact match in the inverted index. Instead, it calculates the edit distance between the search term and the stored tokens, where edit distance refers to the number of single-character insertions, deletions or substitutions needed to transform one string into another.

Storage

Elasticsearch does not store one inverted index per document instead it maintains one inverted index per shard. This inverted index then covers all documents stored in the shard. When a new document is indexed, its tokens are added to the existing shared inverted index.

Apache Lucene write the inverted index to disk in immutable files called segments. Each segment is a small self-contained inverted index. When documents are indexed, new segments are created and merged together in the background. The operating system caches frequently accessed segments in memory. Because of that, Elasticsearch recommends leaving half the available RAM for this file cache rather than allocation it all to the JVM heap.

This design means that the inverted index does consume more storage than the raw documents because it stores the “token-document”-mapping in addition to the original source. However, this trade-off is accepted because the extra storage is what enables near-instant full-text search regardless of how many documents exist.

4.3. Sharding, Replication and Distributed Search

When Elasticsearch receives a search query, it does not search a single location. Instead, a node designated as the coordinating node for that request distributes that query to all shards that belong to the target index, whether they are primary shards or replicas. Each shard executes the search independently and in parallel on its own portion of the data. The shards return their local results along with relevance scores to the coordinating node. The coordinating node then merges all partial results, sorts them by relevance score and returns the final unified response to the client.

5. Key features

5.1. Full-Text Search and Relevance Scoring

Full-Text Search is the most prominent feature of Elasticsearch and the primary reason why organizations adopt to it. Unlike a relational database that performs exact string matching, Elasticsearch analyses both the indexed document and the search query through the analyser pipeline (Section 4.2).

Beyond simply finding matching documents, Elasticsearch assigns each result a relevance score, referred to internally as `_score`. This score reflects how closely a document matches the search query based on factors like how often the search term appears in the document and how rare the term is across the entire index. Results are returned sorted by this score by default, so the most relevant documents appear first. This behaviour is fundamentally different from a relational database, which has no concept of relevance and returns all matching rows equal weight

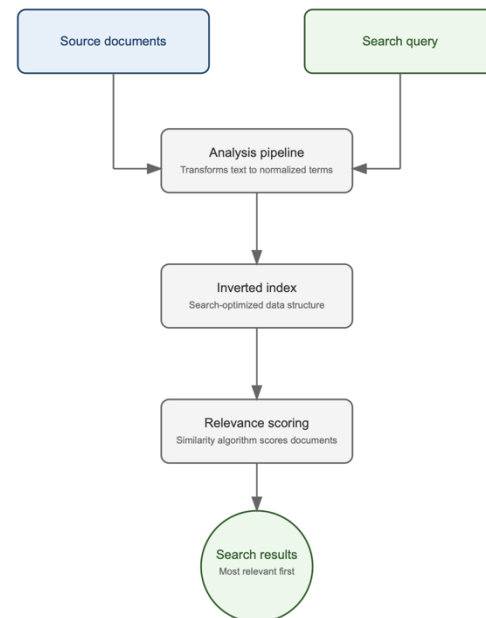


Figure
2: <https://www.elastic.co/docs/solutions/images/elasticsearch-reference-full-text-search-overview.svg>

5.2. Distributed and Scalable

Elasticsearch is designed from the ground up to scale horizontally. When the data volume or query load grows beyond the capacity of a single node, additional nodes can be added to the cluster and Elasticsearch will automatically redistribute shards across them. This means that scaling an Elasticsearch deployment does not require replacing hardware with more powerful machines, it requires adding more machines, which is significantly more cost effective at large scale.

5.3. Near Real-Time Data Analysis

Elasticsearch supports near real-time data indexing, meaning that documents become searchable approximately one second after they are written. This one-second delay results from the internal refresh cycle, during which buffered documents are committed to the index and made available for search. For most applications this delay is imperceptible, and it allows Elasticsearch to handle continuous data streams such as server logs or sensor data.

5.4. RESTful API

All interactions with Elasticsearch are carried out through a RESTful HTTP API. Every operation, indexing a document, executing a search, creating an index or deleting data, is expressed as an HTTP request with a JSON body. This design makes Elasticsearch accessible from any programming language or tool that can send HTTP requests, without requiring a dedicated driver or client library.

Every operation is expressed as a standard HTTP request directed at a specific endpoint. Every response is returned as a JSON object. The four HTTP methods map to the four fundamental operations

- GET → retrieve or search documents
- POST → index a new document or execute a search with a request body
- PUT → create or update an index or document
- DELETE → remove a document or an entire index

For example, searching all documents of the `f1` index looks like this:

```
GET /f1/_search
{
  "query": {
    "match": {
      "race_name": "monaco"
    }
  }
}
```

And the response includes metadata about the query execution alongside the matching documents:

```
{
  "took": 3,
  "hits": {
    "total": { "value": 418 },
    "hits": [
      {
        "_index": "f1",
        "_id": "1",
        "_score": 4.21,
        "_source": {
          "race_name": "Monaco Grand Prix",
          "season": 2023,
          "circuit_country": "Monaco",
          "driver_surname": "Verstappen",
          "finish_position": 1,
          "points": 25.0
        }
      },
      {
        "_index": "f1",
        "_id": "2",
        "_score": 4.21,
        "_source": {
          "race_name": "Monaco Grand Prix",
          "season": 2023,
          "circuit_country": "Monaco",
          "driver_surname": "Alonso",
          "finish_position": 2,
          "points": 18.0
        }
      }
    ]
  }
}
```

The `took` field indicates how many milliseconds the query required. The `_score` field represents the relevance score assigned to each result. The `_source` field contains the original document as it was indexed.

6. Query Language

6.1. Query DSL

The Query DSL is a JSON-based language for building search queries. It is sent as the body of a HTTP Request to the `_search` endpoint. The Query DSL offers different types to search for documents

Match performs a full-text search on a text field.

```
response = es.search(index=ES_INDEX, body={
  "query": {
    "match": {
      "race_name": name
    }
  }
})
```

Fuzzy-search extends the match query by tolerating typos. The `fuzziness: AUTO` setting allows Elasticsearch to calculate the edit distance between the search term and the stored tokens and returns results within the allowed threshold.

```
response = es.search(index=ES_INDEX, body={
  "query": {
    "fuzzy": {
      "driver_surname": {
        "value": surname,
        "fuzziness": "AUTO" # AUTO: allows 1-2 character edits
      }
    },
  },
})
```

Term performs an exact match on a keyword field without any text analysis. It is used for filtering rather than searching.

```
response = es.search(index=ES_INDEX, body={
  "query": {
    "term": {
      "circuit_country": country
    }
  },
})
```

Bool queries combine multiple conditions into a single query using clauses. The must clause requires a condition to match and affects the relevance score. The filter clause also requires a condition to match but does not affect scoring and is cached for performance.

```
response = es.search(index=ES_INDEX, body={
    "query": {
        "bool": {
            "must": [
                {"match": {"driver_surname": surname}} # full-text, affects score
            ],
            "filter": [
                {"range": {"season": {"gte": min_season}}}, # numeric range filter
                {"range": {"finish_position": {"lte": max_position}}}
            ]
        }
    },
})
```

6.2. Aggregations

Aggregations are Elasticsearch's equivalent of SQL Group by combined with aggregate functions such as COUNT, SUM and AVG. Aggregations are defined inside the aggs key of a request body and can be nested to produce multi-level grouping.

```
response = es.search(index=ES_INDEX, body={
    "size": 0,
    "query": {
        "term": {"finish_position": 1} # only count race wins
    },
    "aggs": {
        "wins_per_driver": {
            "terms": {
                "field": "driver_surname.keyword",
                "size": top_n,
                "order": {"_count": "desc"}
            }
        }
    }
})
```

The `"size": 0` tells Elasticsearch not to return any documents hits, only the aggregation result is needed.

7. Project

7.1. Project Description

To demonstrate the core capabilities of Elasticsearch in a practical context, I developed a project using the [F1 World Championship dataset form Kaggle](#). The dataset covers every race result from 1950 until 2024. It consists of several CSV files containing information about races, drivers, constructors, circuits, race results and more.

The goal of the project is to demonstrate how Elasticsearch handles real-world data from ingestion and indexing through to querying and visualization. The project consists of two Python script and a Kibana dashboard, all running against a local Elasticsearch instance inside Docker.

The first script, `f1_dataset_inserter.py`, is responsible for loading the raw csv files. It transforms them into a flat document structure and indexing them into Elasticsearch. The second script, `f1_queries.py`, demonstrates eight different query types against the indexed data, covering full-text search, fuzzy search, exact filtering, Boolean queries and aggregations. The Kibana dashboard provides visual insights into the dataset through charts and an interactive world map of Formula 1 circuits.

The complete project, including both Python scripts, the Docker configuration etc. is available on GitHub at https://github.com/fabi2k03/dab_elasticsearch_paper.

7.2. The Indexing script: `f1_dataset_inserter.py`

The indexing script is the entry point of the project. Its purpose is to take the raw csv files and load them into the Elasticsearch instance.

Loading the csv files

The first step is to load all csv files into pandas dataframes. Each file is read independently and stored. At this stage the data is still in its original form. Each file represents one entity and references other entities only through integer IDS, exactly as a relational database would

Denormalization and document creation

The most important step is the `build_document` function, which merges all six dataframes into a single flat structure. With this step the relational aspect of the data is removed, and the data is combined into one document per race result. The result is one document per race result per driver, meaning each document represents exactly one driver's participation in one race. The document has all associated information such as the race name, circuit, constructor and finishing position. The fields contained in each document correspond exactly to the mapping defined at the top of the script, which specifies the data type of every field and determines how Elasticsearch stores and indexes the data.

Creating the index

Before documents can be indexed, the `create_index` function deletes any existing index with the same name and creates a fresh one with the explicit mapping.

7.3. The Query Script: `f1_queries.py`

The query script demonstrates eight different query types against the indexed F1 data. Each function sends a Query DSL request to Elasticsearch via the Python client and prints the result. The following query types are implemented in the script:

Query	Description
Full-text search	Search races by name using tokenized text matching
Fuzzy search	Find driver even when the name contains one or more typos
Filter	Exact match one keyword field such as country
Bool query	Combine multiple conditions using must and filter, e.g. in which races a driver finished under the top 3
Term aggregation	Group and count results, e.g. wins per driver
Sum aggregation	Calculate totals, e.g. points per constructor
Cardinality aggregation	Count distinct values, e.g. races per season
Multi-match search	Search a single term across multiple fields at once

7.4. Setup and Configuration

Development setup with Docker

For local development, both Elasticsearch and Kibana are run as Docker containers. In the docker-compose file, three environment variables configure the Elasticsearch container.

`discovery.type=single-node`: It declares the Elasticsearch instance as a single node one. Furthermore, it tells Elasticsearch not to wait for other nodes before starting. Without this setting a single node cluster would remain in a waiting state indefinitely.

`xpack.security.enabled=false`: Disables authentication, which is acceptable for a local development environment but must never be used in production

`ES_JAVA_OPTS=-Xms512m -Xmx512m`: sets the JVM (Java Virtual Machine) heap to 512 MB → sufficient for this project but far below the recommended allocation for production workloads

After that file is configured, we can now start the docker-containers with: `docker-compose up -d`

Elasticsearch becomes available at <http://localhost:9200> and Kibana at <http://localhost:5601>

Production setup

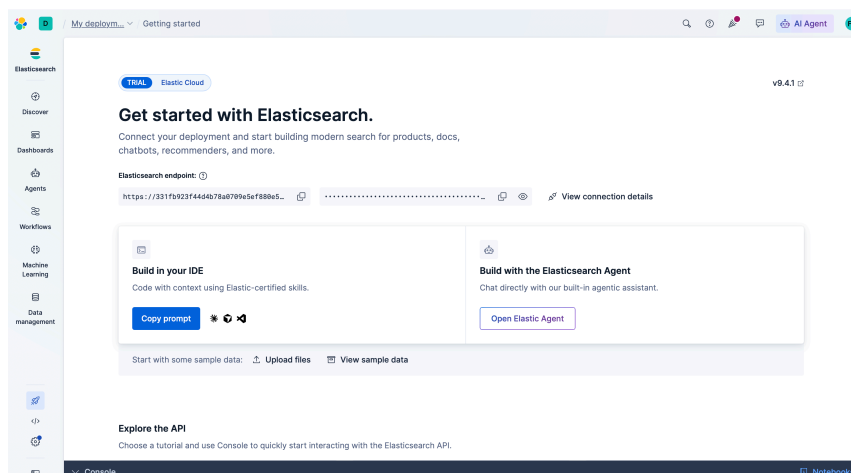
The Docker setup described above is suitable for development and demonstration purposes. A production deployment differs significantly in several aspects.

In production, a cluster should at least consist of three nodes to ensure high availability. Memory allocation is critical, Elasticsearch recommends allocating half of the available system memory to the JVM heap, leaving the remainder for the Lucene file cache. Security must be enabled in production, including TLS encryption and role-based access control.

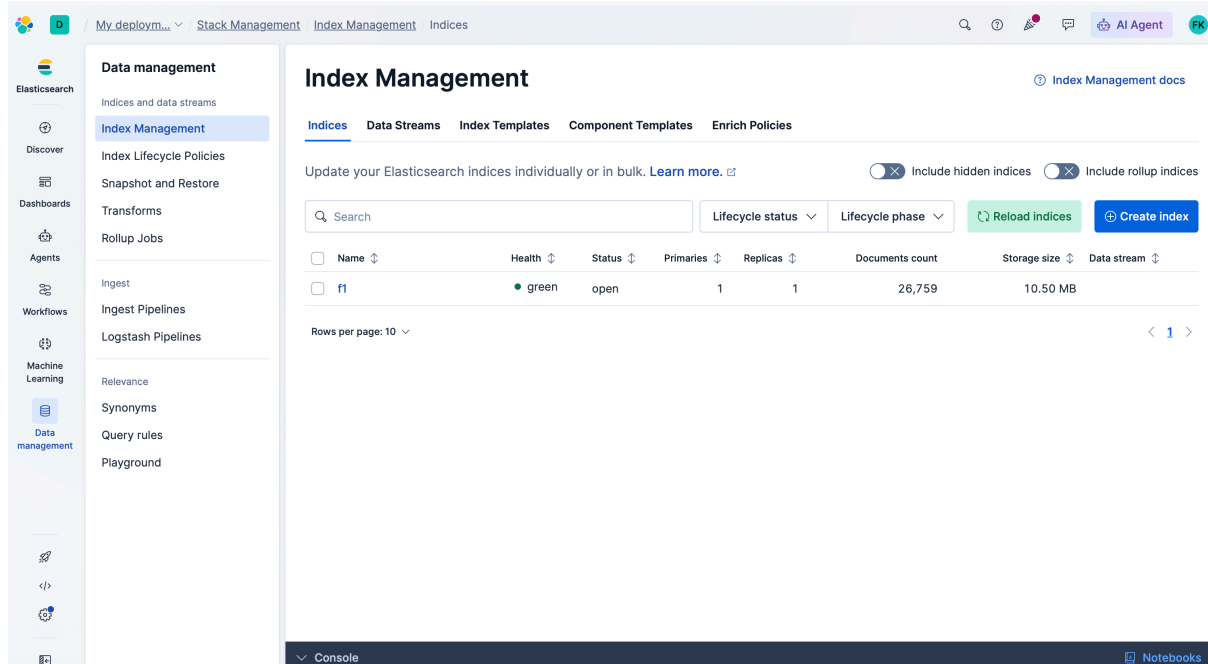
As an alternative to managing the infrastructure manually, Elastic NV offers a managed cloud service called Elastic Cloud. With Elastic Cloud, the entire Elasticsearch and Kibana stack is hosted and maintained by Elastic NV. This eliminates the operational complexity of cluster management, security configuration and software upgrades.

<https://www.elastic.co/de/cloud>

After creating an account and setting up a deployment, Elastic Cloud provides a web-based Kibana interface and a dedicated Elasticsearch endpoint URL. To connect the Python indexing script to the cloud instance, authentication via API key is required. The API key is generated in the Elastic Cloud.



After running the indexing script against the cloud instance, the ‘f1’ index is created with 26,759 documents. The index status is reported as green, indicating that all primary shards and replicas are fully allocated and the cluster is healthy.



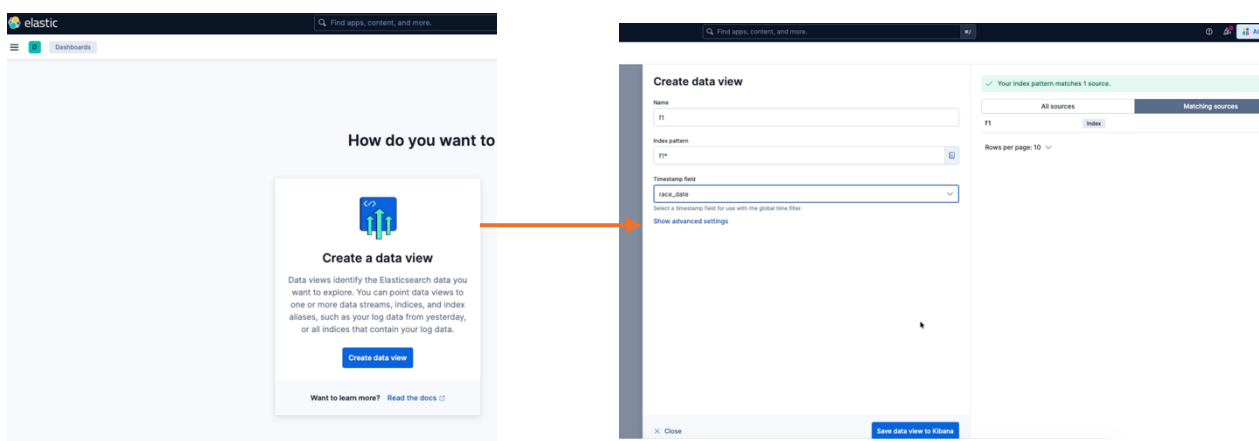
Elastic Cloud is available through a 14-day free trial.

7.5. Kibana Dashboard

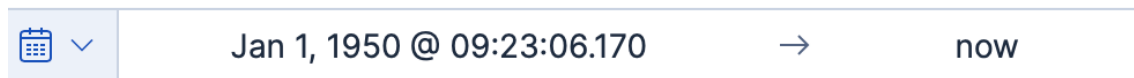
Kibana connects directly to Elasticsearch and provides a web-based interface for creating visualizations and dashboards without writing any code.

To access Kibana, you just have to type in the URL: <http://localhost:5601>.

Before visualizations can be created, a data view must be configured in Kibana that points to the ‘f1’ index and designates ‘_type’ as the time field.



This time field is used by Kibana's default time filter, which is set to "last 15 minutes" by default. For the F1 data, the time range must be adjusted to cover the full period from 1950 to 2024 before any data becomes visible.



The dashboard created for this project contains five visualizations, each demonstrating a different aspect of the indexed data.

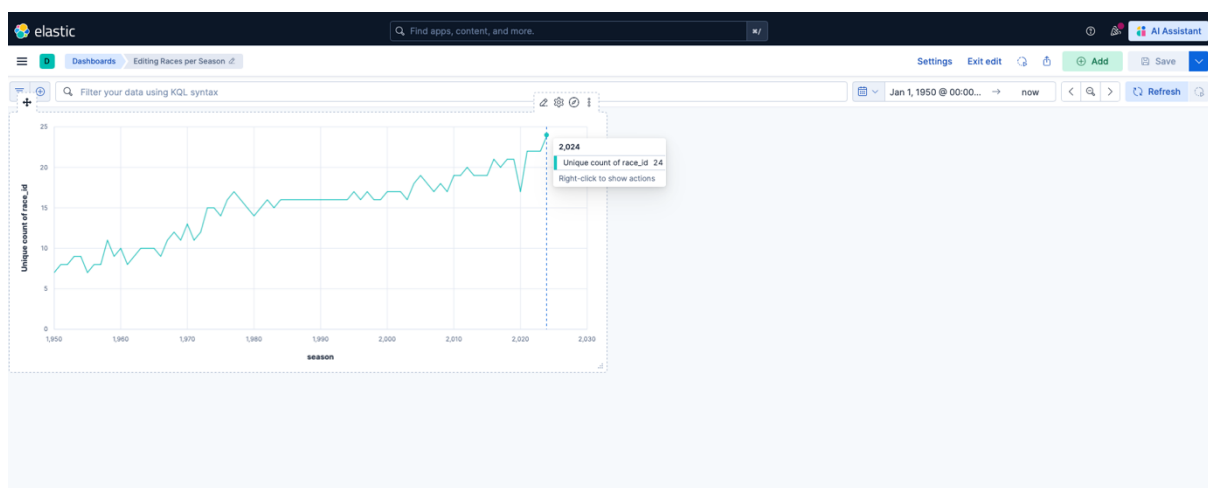
Wins per driver

A bar chart showing the top 10 drivers ranked by total career race wins. It is built by applying a KQL filter of `finish_position: 1` to restrict the dataset to race wins only, then using a unique count of `race_id` per `driver_forename.keyword` and `driver_surname.keyword`.



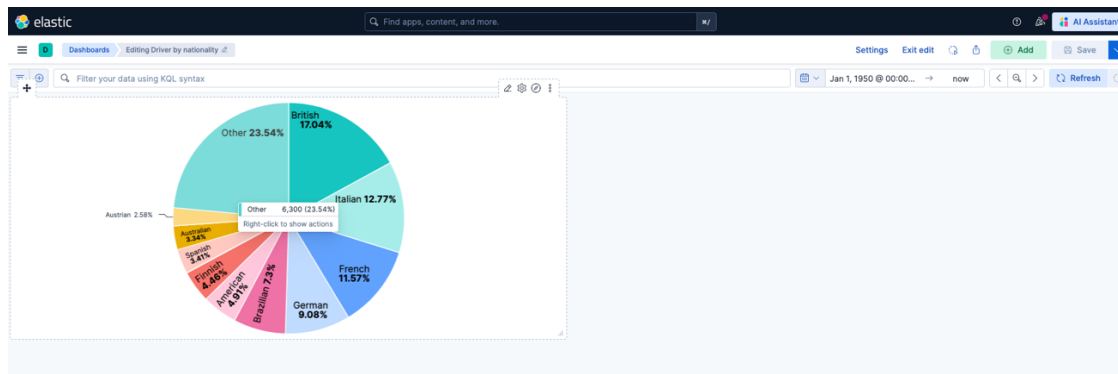
Races per season

Is a line chart showing how many distinct races were held in each Formula 1 season. It uses a terms aggregation on the `season` field combined with a cardinality aggregation on `race_id` to count distinct races per year.



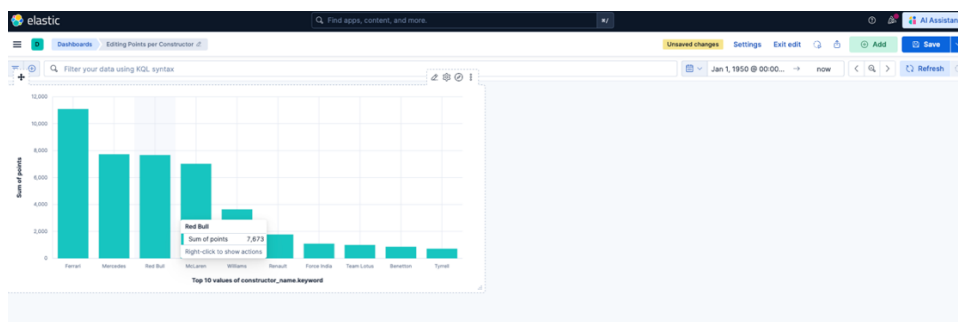
Driver nationalities

Is a pie chart showing the distribution of all Formula 1 drivers by nationality. It uses a term aggregation on the `driver_nationality` keyword field.



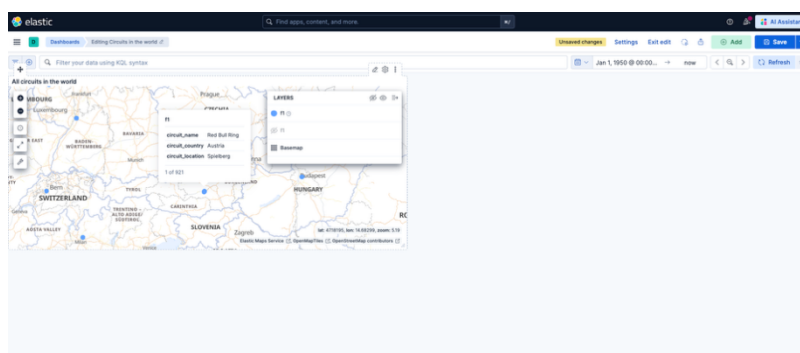
Points per constructor

Is a bar chart showing the total championship points accumulated by each constructor across all seasons. It uses a nested aggregation, a terms aggregation on `constructor_name.keyword` containing a sum aggregation on the `points` field.



Map of F1 circuits

Is a map visualization that plots every F1 track on an interactive world map. This visualization is made possible entirely by the `geo_point` field type defined in the index mapping. Kibana automatically recognizes `circuit_coords` as a geographic coordinate field and renders each unique circuit location as a point on the map. No additional configuration is required beyond selecting the field in the visualization editor.



8. Conclusion

Elasticsearch is a powerful search and analytics engine. Its inverted index structure, distributed architecture and the Query DSL methods make it an excellent choice for applications that require fast full-text search, real-time analytics or geospatial queries over large datasets.

Elasticsearch is not a replacement for relational databases in all scenarios, it lacks ACID transactions and is not suited for use cases where strict data integrity is required. However, for search-heavy or analytics-heavy workloads, it is the best choice.

9. References

- <https://www.geeksforgeeks.org/elasticsearch/what-is-elastic-search-and-why-is-it-used/>
- <https://www.elastic.co/docs/get-started/the-stack#:~:text=The%20Elastic%20Stack%20is%20a,fit%20together%20across%20Elastic%20deployments>
- [Dataset]: <https://www.kaggle.com/datasets/jtrotman/formula-1-race-data>
- <https://www.geeksforgeeks.org/elasticsearch/what-is-elastic-search-and-why-is-it-used/>
- <https://www.geeksforgeeks.org/elasticsearch/elasticsearch-architecture/>
- <https://medium.com/@TechTim42/elastic-search-and-open-search-a-brief-history-of-the-license-war-8f474743e2ff>
- <https://www.cdata.com/blog/elasticsearch-use-cases>
- <https://www.elastic.co/guide/en/elasticsearch/reference/current/analysis.html>
- <https://www.elastic.co/guide/en/elasticsearch/reference/current/documents-indices.html>
- <https://www.elastic.co/docs/solutions/search/full-text/how-full-text-works>
- <https://www.elastic.co/docs/explore-analyze/query-filter/languages/querydsl>
- <https://www.elastic.co/de/cloud>
- <https://www.elastic.co/de/kibana>